

# Documento de Ambiente Técnico:

## CalcEngine Scientific Calculator API

**Author:** Raja SP, AWS Labs

**Source:** <https://github.com/aws-labs/aidlc-workflows>

**Adaptation:** Ricardo de Luna Galdino, EngSoft Learn

**Download:** [example-tech-env-scientific-calculator-api.pt-br.pdf](#)

### Resumo Técnico do Projeto

- **Nome do Projeto:** CalcEngine
  - **Tipo de Projeto:** Greenfield
  - **Ambiente de Execução Principal:** Cloud
  - **Provedor de Cloud:** AWS
  - **Modelo de Implantação Alvo:** Serverless (API Gateway + Lambda)
  - **Gerenciador de Pacotes:** uv
  - **Tamanho do Time:** 4 (2 desenvolvedores backend, 1 desenvolvedor frontend para portal de documentação, 1 engenheiro de QA)
  - **Experiência do Time:** Forte experiência em backend Python, experiência moderada em AWS, sem experiência prévia com desenvolvimento de bibliotecas matemáticas. O time usou FastAPI e Flask profissionalmente. Familiarizado com pytest. Experiência limitada com CDK (precisará de exemplos).
-

# Linguagens de Programação

## Linguagens Obrigatórias

| Linguagem   | Versão  | Finalidade                                                            | Justificativa                                                                                                                                 |
|-------------|---------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Python      | 3.12+   | Serviço de API, motor matemático, handlers Lambda, infraestrutura CDK | Linguagem principal do time. Rico ecossistema matemático (mpmath, numpy, scipy). uv oferece gerenciamento de dependências rápido e confiável. |
| HTML/CSS/JS | ES2022+ | Portal de documentação (site estático)                                | Frontend mínimo para documentação da API. Nenhum framework necessário; geração estática com templates Jinja2.                                 |

## Linguagens Permitidas

| Linguagem  | Condições de Uso                                                                                                                                                                                                 |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rust       | Aprovado para funções matemáticas de alto desempenho (ex.: parser de expressões) se o desempenho do Python for insuficiente. Requer evidências de profiling antes da adoção. Exposto ao Python via PyO3/maturin. |
| TypeScript | Aprovado para infraestrutura CDK se o time preferir CDK em TypeScript ao invés de Python CDK. A decisão deve ser tomada antes do início da construção, não no meio do projeto.                                   |

## Linguagens Proibidas

| Linguagem | Motivo                                                                                | Use em vez disso                                  |
|-----------|---------------------------------------------------------------------------------------|---------------------------------------------------|
| Java      | Sem expertise no time. Adiciona complexidade operacional (cold starts JVM no Lambda). | Python                                            |
| Go        | Sem expertise no time. Python cobre todos os requisitos atuais.                       | Python                                            |
| C/C++     | Carga de manutenção para extensões nativas.                                           | Rust via PyO3 se desempenho nativo for necessário |

## Gerenciamento de Pacotes e Ambiente

### uv como Ferramenta Padrão

uv é a **única ferramenta de gerenciamento de pacotes e ambiente** para este projeto. Não use pip, pip-tools, poetry, pipenv ou conda.

## Padrões de Uso do uv

```
# Inicialização do projeto (já realizada; não execute novamente)
uv init calcengine
cd calcengine

# Adicionando dependências
uv add fastapi                # Adicionar dependência de runtime
uv add uvicorn[standard]      # Adicionar com extras
uv add --dev pytest pytest-cov # Adicionar dependência de desenvolvimento
uv add --dev mypy ruff         # Adicionar ferramentas de desenvolvimento

# Removendo dependências
uv remove requests            # Remover uma dependência

# Executando comandos no ambiente do projeto
uv run python -m calcengine.main # Executar aplicação
uv run pytest                    # Executar testes
uv run mypy src/                 # Executar verificador de tipos
uv run ruff check src/           # Executar linter

# Sincronizando ambiente a partir do lockfile
uv sync                        # Instalar todas as dependências do uv.lock
uv sync --dev                  # Incluir dependências de desenvolvimento

# Gerenciamento do lockfile
# uv.lock é gerado automaticamente. NUNCA edite manualmente.
# uv.lock DEVE ser commitado no controle de versão.
```

## Padrões de Arquivos de Dependência

| Arquivo                      | Finalidade                                                                    | Commitado no Git |
|------------------------------|-------------------------------------------------------------------------------|------------------|
| <code>pyproject.toml</code>  | Metadados do projeto, declarações de dependência, configuração de ferramentas | Sim              |
| <code>uv.lock</code>         | Lockfile determinístico com versões exatas resolvidas                         | Sim              |
| <code>.python-version</code> | Fixar a versão do Python para o projeto (ex.: <code>3.12</code> )             | Sim              |

## Convenções do pyproject.toml

Toda a configuração do projeto fica no `pyproject.toml`. Não crie arquivos de configuração separados para ferramentas que suportam configuração via `pyproject.toml`.

```
[project]
name = "calcengine"
version = "0.1.0"
description = "Scientific calculator REST API"
requires-python = ">=3.12"
dependencies = [
    # Dependências de runtime listadas aqui pelo uv add
]

[dependency-groups]
dev = [
    # Dependências de desenvolvimento listadas aqui pelo uv add --dev
]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --tb=short --strict-markers"
markers = [
    "unit: Unit tests (fast, no external dependencies)",
    "integration: Integration tests (may require services)",
    "accuracy: Mathematical accuracy validation tests",
]

[tool.mypy]
python_version = "3.12"
strict = true
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true

[tool.ruff]
target-version = "py312"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "A", "SIM", "TCH"]

[tool.coverage.run]
source = ["src/calcengine"]
branch = true

[tool.coverage.report]
fail_under = 90
show_missing = true
```

# Frameworks e Bibliotecas

## Frameworks Obrigatórios

| Framework/Biblioteca | Versão | Domínio                                                          | Justificativa                                                                                                    |
|----------------------|--------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| FastAPI              | 0.115+ | Framework REST API                                               | Suporte assíncrono, geração automática de spec OpenAPI, validação Pydantic, forte integração com tipagem Python. |
| Pydantic             | 2.x    | Validação de requisição/resposta, gerenciamento de configurações | Modelos de dados type-safe, serialização JSON, integral ao FastAPI.                                              |
| uvicorn              | 0.30+  | Servidor ASGI                                                    | Servidor de produção padrão para FastAPI. Usado localmente e no Lambda via Mangum.                               |
| Mangum               | 1.x    | Adaptador Lambda                                                 | Envolve o app FastAPI ASGI para o handler AWS Lambda. Adaptador zero-config.                                     |
| pytest               | 8.x    | Framework de testes                                              | Padrão do time. Rico ecossistema de plugins.                                                                     |
| mypy                 | 1.x    | Verificação estática de tipos                                    | Capturar erros de tipo antes do runtime. Modo estrito aplicado.                                                  |
| ruff                 | 0.8+   | Linting e formatação                                             | Substitui flake8, isort e black em uma única ferramenta rápida.                                                  |

| Framework/Biblioteca | Versão | Domínio                    | Justificativa                                                                                                                    |
|----------------------|--------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| structlog            | 24.x+  | Logging JSON estruturado   | Todos os handlers Lambda e endpoints de API devem emitir logs JSON estruturados. Configurado uma vez em um módulo compartilhado. |
| aws-cdk-lib          | 2.x    | Infraestrutura como Código | Implantação AWS. Constructs CDK Python para toda a infraestrutura.                                                               |

## Bibliotecas Preferenciais

Use estas quando sua funcionalidade for necessária. Não as adicione preventivamente.

| Biblioteca     | Finalidade                                      | Usar Quando                                                                                                                                             |
|----------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| mpmath         | Aritmética de precisão arbitrária               | Fase 2: quando o modo de precisão arbitrária for implementado. Não necessário para o MVP (dupla IEEE 754 é suficiente).                                 |
| numpy          | Operações com arrays, álgebra linear            | Fase 2: quando operações de matriz/vetor forem implementadas. Não usar para aritmética básica.                                                          |
| scipy          | Distribuições estatísticas, integração numérica | Fase 2+: quando estatísticas avançadas e cálculo forem implementados.                                                                                   |
| httpx          | Cliente HTTP assíncrono                         | Chamadas HTTP de saída (ex.: busca de taxas de câmbio na Fase 3). Preferido em relação ao requests por compatibilidade assíncrona.                      |
| boto3          | AWS SDK                                         | Qualquer interação direta com serviços AWS não tratada pelo CDK no momento do deploy (ex.: consultas DynamoDB, leituras do Secrets Manager em runtime). |
| pytest-cov     | Relatório de cobertura de testes                | Sempre. Incluído nas dependências de desenvolvimento desde o início do projeto.                                                                         |
| pytest-asyncio | Suporte a testes assíncronos                    | Ao testar endpoints FastAPI assíncronos ou funções assíncronas.                                                                                         |
| hypothesis     | Testes baseados em propriedades                 | Testes de funções matemáticas. Gera entradas aleatórias para encontrar casos extremos. Fortemente recomendado para todos os módulos matemáticos.        |
| freezegun      | Mock de tempo                                   | Ao testar lógica dependente de tempo (rate limiting, expiração de tokens, timestamps de auditoria).                                                     |



## Bibliotecas Proibidas

| Biblioteca                  | Motivo                                                                                    | Alternativa                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Flask                       | O projeto usa FastAPI. Não misture frameworks web.                                        | FastAPI                                                                                               |
| Django                      | Excessivo para um serviço de API. ORM não é necessário.                                   | FastAPI + acesso direto ao DynamoDB                                                                   |
| requests                    | Somente síncrono. Bloqueia o event loop assíncrono no FastAPI.                            | httpx                                                                                                 |
| sympy                       | Muito pesado para o escopo do MVP. Puxa uma grande árvore de dependências.                | Implemente o parser de expressões diretamente. Reavalie para computação simbólica na Fase 3.          |
| pandas                      | Não necessário. CalcEngine processa cálculos individuais, não dataframes.                 | Python padrão ou numpy para operações com arrays quando necessário.                                   |
| SQLAlchemy                  | Sem banco de dados relacional no MVP. DynamoDB é o armazenamento de dados.                | boto3 DynamoDB resource/client                                                                        |
| celery                      | Complexidade desnecessária para o MVP. Todos os cálculos são síncronos e rápidos (<50ms). | Reavaliar na Fase 3 para processamento em lote. Usar SQS + Lambda se assíncrono for necessário antes. |
| poetry / pipenv / pip-tools | O projeto usa uv exclusivamente. Não introduza gerenciadores de pacotes alternativos.     | uv                                                                                                    |
| black / isort / flake8      | Substituídos pelo ruff, que combina os três.                                              | ruff                                                                                                  |

## Processo de Aprovação de Bibliotecas

Para adicionar uma biblioteca que não está nas listas obrigatória ou preferencial:

1. Abra uma issue no GitHub com o título "Dependency Request: [nome-da-biblioteca]"
  2. Inclua: finalidade, alternativas consideradas, licença (deve ser MIT, Apache 2.0 ou BSD), status de manutenção (data do último lançamento, contagem de issues abertas) e impacto no tamanho
  3. O tech lead revisa e aprova ou rejeita
  4. Se aprovado, adicione via `uv add` e atualize este documento
- 

## Ambiente Cloud

### Provedor de Cloud

- **Provedor Principal:** AWS
- **Estrutura de Conta:** Conta AWS única para o MVP. Contas separadas dev/staging/prod na Fase 2.
- **Regiões:** `us-east-1` (principal). Sem região de recuperação de desastres para o MVP. Multi-região planejado para a Fase 2.

## Lista de Serviços Permitidos

| Serviço                       | Casos de Uso Aprovados                                                                   | Restrições                                                                                                                           |
|-------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| AWS Lambda                    | Handlers de requisição de API, computação matemática                                     | Runtime Python 3.12. Máximo 256MB de memória para o MVP (aumentar se profiling indicar necessidade). Timeout de 30 segundos.         |
| Amazon API Gateway (HTTP API) | Endpoint público de REST API                                                             | Tipo HTTP API (não tipo REST API). Domínio customizado com TLS. Planos de uso para rate limiting.                                    |
| Amazon DynamoDB               | Armazenamento de chaves de API, medição de uso, contadores de rate limit                 | Modo de capacidade sob demanda. Design de tabela única. TTL para janelas de rate limit.                                              |
| Amazon S3                     | Hospedagem da spec OpenAPI, site de documentação estático, pacotes de implantação Lambda | Criptografia de bucket habilitada. Acesso público bloqueado, exceto para o bucket do site de documentação (distribuição CloudFront). |
| Amazon CloudFront             | CDN para portal de documentação e spec de API                                            | Somente HTTPS. Cache agressivo de ativos estáticos.                                                                                  |
| Amazon CloudWatch             | Logging, métricas, alarmes, dashboards                                                   | Logs JSON estruturados de todos os Lambdas. Métricas customizadas para contagens de cálculos, percentis de latência, taxas de erro.  |
| AWS Secrets Manager           | Chaves de API do Stripe, chaves de assinatura                                            | Rotação automática onde suportado. Lambda lê no cold start, armazena em memória.                                                     |
| AWS Certificate Manager       | Certificados TLS para domínio customizado                                                | Usado com API Gateway e CloudFront.                                                                                                  |

| Serviço        | Casos de Uso Aprovados                                                                               | Restrições                                                                                                                       |
|----------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Amazon Cognito | Autenticação de contas de desenvolvedor para portal de documentação e gerenciamento de chaves de API | Pool de usuários para signup/login de desenvolvedores. Não usado para autenticação de chamadas de API (chaves de API para isso). |
| Amazon SQS     | Fila de dead-letter para operações assíncronas com falha                                             | Fila padrão. Usada para eventos de faturamento com falha e captura de erros. Não usada para requisições de cálculo no MVP.       |
| AWS CDK        | Implantação de Infraestrutura como Código                                                            | CDK Python. Toda infraestrutura definida no CDK. Sem alterações manuais no console.                                              |
| AWS CloudTrail | Logging de auditoria de API                                                                          | Habilitado para todos os eventos de gerenciamento. Eventos de dados para S3 e Lambda em produção.                                |
| AWS IAM        | Permissões de serviço                                                                                | Políticas de menor privilégio por função Lambda. Sem permissões de recurso com wildcard.                                         |

## Lista de Serviços Não Permitidos

| Serviço                    | Motivo                                                                                   | Alternativa                                                                 |
|----------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Amazon EC2                 | Sobrecarga operacional. Modelo serverless preferido.                                     | Lambda para computação.                                                     |
| Amazon ECS / Fargate       | Engenharia excessiva para carga de trabalho de requisição/resposta do MVP.               | Lambda. Reavaliar se cold starts se tornarem um problema.                   |
| Amazon RDS / Aurora        | Banco de dados relacional não necessário. Dados de chave de API e uso cabem no DynamoDB. | DynamoDB.                                                                   |
| Amazon ElastiCache / Redis | Sem camada de cache necessária para o MVP. Cálculos são stateless e rápidos.             | Cache em memória dentro do contexto de execução Lambda se necessário.       |
| AWS Elastic Beanstalk      | Não se adequa ao modelo IaC.                                                             | CDK + Lambda.                                                               |
| Amazon Kinesis             | Streaming não necessário. Todos os cálculos são requisição/resposta síncronos.           | SQS se processamento assíncrono for necessário.                             |
| AWS Step Functions         | Sem orquestração de múltiplos passos no MVP.                                             | Invocação direta do Lambda. Reavaliar para processamento em lote na Fase 3. |
| Amazon SNS                 | Sem pub/sub necessário no MVP.                                                           | SQS para filas dead-letter.                                                 |

## Processo de Aprovação de Serviços

Para usar um serviço que não está na lista de permitidos:

1. Abra uma issue no GitHub com o título "AWS Service Request: [nome-do-serviço]"
2. Inclua: caso de uso, estimativa de custo (mensal), implicações de segurança, carga operacional e por que um serviço permitido não pode atender à necessidade

3. O tech lead revisa. Serviços com acesso a PII ou exposição de rede requerem revisão de segurança adicional.
4. Se aprovado, adicione o construct CDK e atualize este documento

## Tecnologias e Padrões Preferenciais

### Padrão de Arquitetura

#### Monólito modular implantado como funções serverless.

CalcEngine é um único pacote Python com módulos internos (aritmética, trigonometria, estatística, etc.), exposto através de uma única aplicação FastAPI, implantado no AWS Lambda por trás do API Gateway. Esta não é uma arquitetura de microsserviços.

| Decisão               | Escolha                                                       | Justificativa                                                                                                                                                                                  |
|-----------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Estilo de arquitetura | Monólito modular                                              | Time pequeno (4 pessoas), domínio único, sem requisitos de escalabilidade independente por módulo no MVP.                                                                                      |
| Modelo de implantação | Função Lambda única servindo todas as rotas de API via Mangum | Simplicidade. Um artefato de implantação. Cold start amortizado em todos os endpoints.                                                                                                         |
| Fronteiras de módulo  | Pacotes Python dentro de <code>src/calccengine/</code>        | Fronteiras internas limpas sem o custo operacional de serviços separados. Pode extrair para Lambdas separados posteriormente se endpoints específicos precisarem de memória/timeout diferente. |

### Padrões de Design de API

- **Estilo:** REST sobre HTTPS. Corpos de requisição e resposta JSON.
- **URL Base:** `https://api.calccengine.io/v1/`
- **Versionamento:** Prefixo de caminho de URL (`/v1/`, `/v2/`). Apenas versão principal. Mudanças não disruptivas não incrementam a versão.

- **Documentação:** Especificação OpenAPI 3.1 gerada automaticamente pelo FastAPI. Hospedada em `https://docs.calcengine.io`.
- **Convenção de Nomenclatura:** snake\_case para nomes de campos JSON (convenção Python). kebab-case para caminhos de URL.
- **Tipo de Conteúdo:** `application/json` para todas as requisições e respostas. Sem suporte a XML.

### Formato Padrão de Requisição:

```
{
  "expression": "sin(pi/4) * 2 + sqrt(16)",
  "options": {
    "angle_mode": "radians",
    "precision": 15
  }
}
```

### Formato Padrão de Resposta de Sucesso:

```
{
  "result": 5.414213562373095,
  "expression": "sin(pi/4) * 2 + sqrt(16)",
  "computation_time_ms": 2.3,
  "engine_version": "0.1.0"
}
```

### Formato Padrão de Resposta de Erro:

```
{
  "error": {
    "code": "DOMAIN_ERROR",
    "message": "Cannot compute logarithm of a negative number",
    "detail": "log(-5) is undefined for real numbers",
    "parameter": "expression",
    "documentation_url": "https://docs.calcengine.io/errors/DOMAIN_ERROR"
  }
}
```

### Códigos de Erro (MVP):

| Código                           | Status HTTP | Significado                                                                |
|----------------------------------|-------------|----------------------------------------------------------------------------|
| <code>PARSE_ERROR</code>         | 400         | Expressão não pôde ser parseada. Sintaxe malformada.                       |
| <code>DOMAIN_ERROR</code>        | 422         | Matematicamente indefinido ( $\log(-1)$ , $\sqrt{-1}$ , divisão por zero). |
| <code>OVERFLOW_ERROR</code>      | 422         | Resultado excede o intervalo representável.                                |
| <code>INVALID_PARAMETER</code>   | 400         | Parâmetro de requisição tem tipo ou valor inválido.                        |
| <code>EXPRESSION_TOO_LONG</code> | 400         | Expressão excede o comprimento máximo permitido.                           |
| <code>RATE_LIMIT_EXCEEDED</code> | 429         | Chave de API excedeu seu rate limit.                                       |
| <code>UNAUTHORIZED</code>        | 401         | Chave de API ausente ou inválida.                                          |
| <code>INTERNAL_ERROR</code>      | 500         | Erro inesperado no servidor.                                               |

## Padrões de Dados

- **Armazenamento de Dados Principal:** DynamoDB (design de tabela única)
- **Entidades no DynamoDB:** Chaves de API, contadores de uso (por chave por mês), janelas de rate limit (por chave por minuto)
- **Padrão de Acesso:** Todas as leituras e escritas são por chave primária (ID da chave de API). Sem scans. Sem consultas complexas.
- **Cache:** Sem cache externo. Lambda reutiliza conexões DynamoDB entre invocações quentes. Resultados de validação de chave de API armazenados em memória Lambda por 60 segundos.
- **Sem banco de dados relacional:** Se consultas relacionais se tornarem necessárias (relatórios, análises), avalie exportação do DynamoDB para S3 + Athena antes de adicionar RDS.

## Padrão de Logging

Toda saída de log deve ser JSON estruturado via structlog. Saída legível por humanos no console apenas para desenvolvimento local.



```

import structlog

logger = structlog.get_logger()

# Chamada de log padrão
logger.info(
    "calculation_completed",
    expression=expression,
    result=result,
    computation_time_ms=elapsed,
    api_key_id=api_key_id,
)

# Chamada de log de erro
logger.error(
    "calculation_failed",
    expression=expression,
    error_code="DOMAIN_ERROR",
    error_detail=str(e),
    api_key_id=api_key_id,
)

```

### Campos de log obrigatórios para cada requisição de API:

| Campo                    | Descrição                                                             |
|--------------------------|-----------------------------------------------------------------------|
| <code>request_id</code>  | ID único por requisição (do API Gateway ou gerado)                    |
| <code>api_key_id</code>  | Identificador de chave de API com hash (nunca registre a chave bruta) |
| <code>endpoint</code>    | Caminho da API chamado                                                |
| <code>http_method</code> | GET, POST, etc.                                                       |
| <code>http_status</code> | Código de status da resposta                                          |
| <code>duration_ms</code> | Tempo total de processamento da requisição                            |
| <code>timestamp</code>   | Timestamp ISO 8601                                                    |

# Requisitos de Segurança

## Autenticação e Autorização

- **Autenticação de Chamadas de API:** Chave de API passada no cabeçalho `Authorization: Bearer {key}`. Chaves de API são strings aleatórias de 32 caracteres, armazenadas como hashes bcrypt no DynamoDB.
- **Autenticação no Portal do Desenvolvedor:** Pool de usuários Amazon Cognito. Signup com email + senha com verificação de email.
- **Modelo de Autorização:** Plano. Todas as chaves de API têm acesso a todos os endpoints. Rate limits por tier (free, starter, professional) aplicados por medição de uso, não por permissões em nível de endpoint.
- **Gerenciamento de Chaves de API:** Desenvolvedores criam, rotacionam e revogam chaves pelo portal do desenvolvedor. Máximo de 3 chaves ativas por conta.

## Proteção de Dados

- **Criptografia em Repouso:** DynamoDB criptografado com chave KMS gerenciada pela AWS. Buckets S3 criptografados com SSE-S3.
- **Criptografia em Trânsito:** TLS 1.2+ aplicado no domínio customizado do API Gateway e na distribuição CloudFront. Sem endpoints HTTP (texto puro).
- **Tratamento de PII:** Contas de desenvolvedores armazenam email e senha com hash. Nenhum outro PII coletado. Expressões matemáticas não são PII. Expressões são registradas para depuração, mas não armazenadas permanentemente (retenção de logs CloudWatch: 30 dias).
- **Classificação de Dados:** Chaves de API = Confidencial. Emails de desenvolvedores = Interno. Expressões matemáticas e resultados = Público.

## Validação de Entrada

- **Limite de comprimento de expressão:** Máximo de 4.096 caracteres. Rejeite expressões mais longas com `EXPRESSION_TOO_LONG`.
- **Lista de caracteres permitidos em expressão:** Alfanumérico, operadores aritméticos (`+ - * / ^ %`), parênteses, ponto decimal, vírgula, espaço em branco e nomes de funções reconhecidas. Rejeite caracteres não reconhecidos.
- **Sem execução de código:** O parser de expressões nunca deve chamar `eval()`, `exec()`, `compile()` ou qualquer execução dinâmica de código. Expressões são parseadas em um AST e avaliadas pelo motor matemático.

- **Limite de profundidade de recursão:** O parser de expressões limita a profundidade de aninhamento a 100 níveis. Previne estouro de pilha em expressões profundamente aninhadas como `(((((...))))).`
- **Validação de intervalo numérico:** Resultados que excedem o intervalo de dupla precisão IEEE 754 retornam `OVERFLOW_ERROR` ao invés de `Infinity` ou `NaN`.

## Gerenciamento de Segredos

- **Chaves de API do Stripe:** Armazenadas no AWS Secrets Manager. Lidas pelo Lambda no cold start, armazenadas em memória.
- **Segredo do Cliente Cognito:** Armazenado no AWS Secrets Manager.
- **Práticas Proibidas:**
  - Sem segredos no `pyproject.toml`, código-fonte ou arquivos `.env` commitados no Git
  - Sem segredos em variáveis de ambiente Lambda (use Secrets Manager em runtime)
  - Sem chaves de acesso AWS no código (Lambda usa roles de execução IAM)
  - Arquivos `.env` apenas para desenvolvimento local, listados no `.gitignore`

## Segurança de Dependências

- **Varredura:** GitHub Dependabot habilitado para dependências Python. Alertas sobre vulnerabilidades conhecidas.
- **Política de Licenças:** Permitidas: MIT, Apache 2.0, BSD (2-cláusulas e 3-cláusulas), PSF, ISC. Proibidas: GPL, LGPL, AGPL, SSPL, proprietárias. Verifique com `uv tree` antes de adicionar novas dependências.
- **Política de Atualização:** CVEs Críticos/Altos corrigidos em 7 dias. Médios em 30 dias. Baixos avaliados trimestralmente.

## Conformidade OWASP Top 10 (2021)

### A01:2021 - Controle de Acesso Quebrado

| Controle                                     | Implementação CalcEngine                                                                                                                                                                                                                      |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aplicação de autorização                     | Chave de API validada no middleware FastAPI ( <code>api/middleware/auth.py</code> ) em cada requisição antes da execução do handler de rota. Nenhum endpoint é acessível sem uma chave válida.                                                |
| Negação padrão                               | API Gateway rejeita requisições sem cabeçalho <code>Authorization</code> no nível do gateway (401). Handler Lambda rejeita requisições com chaves inválidas ou revogadas (401).                                                               |
| Propriedade de recursos                      | Cada chave de API está vinculada a uma conta Cognito. Desenvolvedores só podem listar, rotacionar e revogar suas próprias chaves. Consultas DynamoDB são limitadas à chave de partição do usuário autenticado.                                |
| Rate limiting                                | Rate limits por chave aplicados no middleware ( <code>api/middleware/rate_limit.py</code> ). Free: 10.000 chamadas/mês, 10 chamadas/segundo. Starter: 1M/mês, 50/segundo. Professional: 10M/mês, 200/segundo. Exceder os limites retorna 429. |
| Política CORS                                | CORS do API Gateway configurado para permitir apenas a origem do portal de documentação ( <code>https://docs.calcengine.io</code> ). Sem origens wildcard. Apenas métodos <code>GET</code> e <code>POST</code> .                              |
| Directory traversal / manipulação de caminho | Não aplicável. CalcEngine não serve arquivos nem aceita caminhos de arquivo como entrada.                                                                                                                                                     |

## A02:2021 - Falhas Criptográficas

| Controle                          | Implementação CalcEngine                                                                                                                                                                                                                         |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dados em trânsito                 | TLS 1.2+ aplicado no domínio customizado do API Gateway e CloudFront. Endpoints HTTP não existem. API Gateway configurado com <code>SecurityPolicy: TLS_1_2</code> .                                                                             |
| Dados em repouso                  | DynamoDB criptografado com chave KMS gerenciada pela AWS. Buckets S3 criptografados com SSE-S3. Logs CloudWatch criptografados com chaves gerenciadas pelo serviço.                                                                              |
| Armazenamento de senha/credencial | Senhas do portal do desenvolvedor com hash via bcrypt (gerenciado pelo Cognito). Chaves de API armazenadas como hashes bcrypt no DynamoDB. Chaves de API brutas são retornadas exatamente uma vez na criação e nunca armazenadas ou registradas. |
| Dados sensíveis em respostas      | Respostas de API nunca contêm chaves de API, credenciais de conta ou identificadores internos. Mensagens de erro não vazam nomes de tabelas, ARNs ou stack traces.                                                                               |
| Dados sensíveis em logs           | IDs de chaves de API (identificador com hash, não a chave em si) são registrados. Chaves de API brutas nunca são registradas. Emails de desenvolvedores não são incluídos em logs de cálculo.                                                    |

## A03:2021 - Injeção

| Controle                       | Implementação CalcEngine                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Injeção de expressão           | O parser de expressões constrói um AST a partir de uma gramática estrita. Ele <b>não</b> usa <code>eval()</code> , <code>exec()</code> , <code>compile()</code> ou qualquer mecanismo de execução de código Python. Apenas tokens reconhecidos (números, operadores, parênteses, nomes de funções na whitelist) são aceitos. Tokens não reconhecidos causam <code>PARSE_ERROR</code> (400).                                                |
| Lista de caracteres permitidos | Entrada de expressão restrita a: dígitos, ponto decimal, operadores aritméticos ( <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code> , <code>%</code> ), parênteses, vírgula, espaço em branco e um conjunto fixo de nomes de funções ( <code>sin</code> , <code>cos</code> , <code>tan</code> , <code>log</code> , <code>sqrt</code> , etc.). Todos os outros caracteres são rejeitados antes do parsing. |
| Injeção NoSQL                  | Consultas DynamoDB usam o SDK boto3 com condições de chave parametrizadas. Sem concatenação de string de entrada do usuário em expressões de consulta. Chaves de partição e chaves de ordenação são definidas programaticamente, nunca interpoladas de corpos de requisição.                                                                                                                                                               |
| Injeção de cabeçalho HTTP      | FastAPI e Pydantic validam e verificam o tipo de toda entrada de requisição. Cabeçalhos de resposta são definidos programaticamente pelo framework, não a partir de entrada do usuário.                                                                                                                                                                                                                                                    |
| Injeção de log                 | structlog escapa caracteres especiais em valores de log. Expressões fornecidas pelo usuário são registradas como valores de string dentro de campos JSON estruturados, não interpoladas em strings de formato de log.                                                                                                                                                                                                                      |

## A04:2021 - Design Inseguro

| Controle                     | Implementação CalcEngine                                                                                                                                                                                                                                                 |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modelagem de ameaças         | Modelo de ameaças criado durante o estágio NFR Requirements do AIDLC. Revisado quando novos endpoints ou pontos de integração são adicionados. Principais ameaças: injeção de expressão, esgotamento de recursos, abuso de chave de API.                                 |
| Defesa em profundidade       | Validação em três camadas: (1) validação de requisição no API Gateway, (2) validação de modelo Pydantic no FastAPI, (3) validação de domínio nas funções do motor. Cada camada rejeita independentemente.                                                                |
| Limites de lógica de negócio | Comprimento de expressão limitado a 4.096 caracteres.<br>Profundidade de recursão do parser limitada a 100 níveis. Tamanho máximo de array para endpoints de estatística: 10.000 elementos.<br>Esses limites previnem esgotamento de recursos sem afetar o uso legítimo. |
| Testes de casos de abuso     | A suite de testes inclui testes negativos/de abuso: expressões sobredimensionadas, parênteses profundamente aninhados, expressões projetadas para causar avaliação lenta, requisições em rajada excedendo rate limits, chaves de API inválidas/expiradas/revogadas.      |

## A05:2021 - Configuração de Segurança Incorreta

| Controle                       | Implementação CalcEngine                                                                                                                                                                                                                                                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Infraestrutura como Código     | Toda infraestrutura definida no AWS CDK (Python). Sem alterações manuais no console. CDK diff revisado em pull requests antes do deploy.                                                                                                                                                                                            |
| Credenciais padrão             | Sem chaves de API padrão, contas de administrador ou senhas fixas em nenhum ambiente. Pool de usuários Cognito requer verificação de email.                                                                                                                                                                                         |
| Mensagens de erro              | Respostas de erro em produção retornam o código de erro CalcEngine, uma mensagem amigável ao usuário e uma URL de documentação. Nunca expõem tracebacks Python, ARNs Lambda, nomes de tabelas DynamoDB ou caminhos de arquivo internos. FastAPI <code>debug=False</code> em produção.                                               |
| Funcionalidades desnecessárias | Sem endpoints interativos <code>/docs</code> ou <code>/redoc</code> expostos no Lambda de produção. Spec OpenAPI servida apenas do site de documentação estático. Sem endpoints de health-check que revelem detalhes de versão além de <code>engine_version</code> .                                                                |
| Cabeçalhos de segurança        | Respostas do API Gateway incluem: <code>Strict-Transport-Security: max-age=31536000; includeSubDomains</code> , <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Cache-Control: no-store</code> nas respostas de API. CloudFront adiciona cabeçalhos de segurança ao site de documentação. |
| Configuração Lambda            | Funções Lambda usam a memória mínima necessária (256MB). Timeout definido para 30 segundos. Concorrência reservada configurada para evitar escalabilidade descontrolada. Sem variáveis de ambiente contendo segredos (Secrets Manager em runtime).                                                                                  |



## A06:2021 - Componentes Vulneráveis e Desatualizados

| Controle                  | Implementação CalcEngine                                                                                                                                                                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Varredura de dependências | GitHub Dependabot habilitado. Verifica <code>pyproject.toml</code> e <code>uv.lock</code> por vulnerabilidades conhecidas. Alertas criam issues no GitHub automaticamente.                    |
| SLA de correção           | CVEs Críticos/Altos: corrigidos em 7 dias. Médios: 30 dias. Baixos: avaliados trimestralmente.                                                                                                |
| Conformidade de licença   | Permitidas: MIT, Apache 2.0, BSD, PSF, ISC. Proibidas: GPL, LGPL, AGPL, SSPL, proprietárias. Verificado com <code>uv tree</code> antes de adicionar dependências.                             |
| Integridade do lockfile   | <code>uv.lock</code> commitado no Git e aplicado no CI. <code>uv sync --locked</code> no pipeline CI falha se o lockfile estiver desatualizado. Sem <code>uv add</code> ad-hoc no CI.         |
| Dependências mínimas      | A lista de bibliotecas proibidas previne árvores de dependências infladas (sem pandas, Django, SQLAlchemy, sympy no MVP). Cada nova dependência requer uma issue no GitHub com justificativa. |

## A07:2021 - Falhas de Identificação e Autenticação

| Controle                                | Implementação CalcEngine                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hash de chaves de API                   | Chaves de API são strings aleatórias criptograficamente seguras de 32 caracteres (via <code>secrets.token_urlsafe</code> ). Armazenadas como hashes bcrypt. A pesquisa usa um prefixo de chave (primeiros 8 chars, armazenados em texto puro) para encontrar o registro, depois a verificação bcrypt confirma a chave completa. |
| Proteção contra força bruta             | Throttling do API Gateway: 100 requisições/segundo por IP em todos os endpoints. Tentativas de autenticação com falha (chave inválida) registradas com <code>api_key_prefix</code> e IP de origem. Após 50 tentativas de autenticação com falha de um único IP em 5 minutos, bloqueio temporário de IP via regra WAF.           |
| Autenticação no portal do desenvolvedor | Cognito aplica: senha mínima de 12 caracteres, verificação de email obrigatória, bloqueio de conta após 5 tentativas de login com falha.                                                                                                                                                                                        |
| Rotação de chaves                       | Desenvolvedores podem criar uma nova chave antes de revogar a antiga (período de sobreposição para rotação sem downtime). Máximo de 3 chaves ativas por conta previne acúmulo de chaves.                                                                                                                                        |
| Exposição de credenciais                | Chave de API retornada exatamente uma vez na criação (no corpo da resposta HTTP). Não armazenada em texto puro em lugar algum. Não incluída em emails. Não visível no portal do desenvolvedor após a criação.                                                                                                                   |
| Autenticação multifator                 | Não obrigatória para o MVP. Suporte a MFA do Cognito está disponível e será habilitado como opção na Fase 2 quando contas de time/empresa forem introduzidas.                                                                                                                                                                   |

## A08:2021 - Falhas de Integridade de Software e Dados

| Controle                               | Implementação CalcEngine                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Segurança do pipeline CI/CD            | GitHub Actions. Branch <code>main</code> protegida: requer PR, pelo menos 1 revisão, todas as verificações CI passando. Sem pushes diretos para <code>main</code> . Workflow de deploy acionado apenas no merge para <code>main</code> .                                                                                                       |
| Integridade de dependências            | <code>uv.lock</code> contém hashes para todas as dependências. <code>uv sync --locked</code> verifica hashes na instalação. Mudanças no lockfile em PRs são revisadas explicitamente.                                                                                                                                                          |
| Integridade do artefato de implantação | Pacote de implantação Lambda construído no CI a partir de uma instalação limpa <code>uv sync --locked</code> . Sem builds locais implantados em produção. CDK deploy executa apenas do pipeline CI, não de máquinas de desenvolvedores.                                                                                                        |
| Segurança de desserialização           | Modelos Pydantic v2 fazem parse e validam todo JSON de entrada. Sem uso de <code>pickle</code> , <code>yaml.load()</code> (loader inseguro) ou <code>marshal</code> . Apenas <code>json.loads()</code> via parsing JSON do Pydantic. <code>model_config</code> do Pydantic tem <code>extra = "forbid"</code> para rejeitar campos inesperados. |

## A09:2021 - Falhas de Logging e Monitoramento de Segurança

| Controle                         | Implementação CalcEngine                                                                                                                                                                                                                                                                               |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Eventos de segurança registrados | Todos os eventos abaixo são registrados como JSON estruturado no CloudWatch: falhas de autenticação (chave inválida/expirada/revogada), rate limit excedido (429), falhas de validação de entrada (400), anomalias de autorização e todos os erros 5xx.                                                |
| Proteção de logs                 | Logs CloudWatch retidos por 30 dias. Política de recurso do grupo de logs impede exclusão pela role de execução Lambda. Logs CloudTrail registram eventos de gerenciamento em um bucket S3 separado com object lock.                                                                                   |
| Alertas                          | CloudWatch Alarms configurados para: taxa de erros 5xx > 1% em 5 minutos, taxa de falha de autenticação > 100/minuto, chave de API única gerando > 10x seu rate limit em tentativas, execução concorrente Lambda > 80% da concorrência reservada. Alarmes notificam via SNS para email/SMS de plantão. |
| Dashboard de monitoramento       | Dashboard CloudWatch exibe: contagem de requisições, taxa de erros (4xx e 5xx), latência p50/p95/p99, contagem de falhas de autenticação, contagem de rate limit atingido, percentual de cold start Lambda, capacidade consumida DynamoDB. Revisado semanalmente.                                      |

## A10:2021 - Server-Side Request Forgery (SSRF)

| Controle             | Implementação CalcEngine                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aplicabilidade       | <b>Baixo risco para o MVP.</b> CalcEngine não faz requisições HTTP de saída baseadas em entrada do usuário. O parser de expressões avalia expressões matemáticas; não busca URLs, resolve hostnames ou faz chamadas de rede.                                                                                                                                                                         |
| Requisições de saída | As únicas chamadas de rede de saída do Lambda são: (1) consultas DynamoDB via AWS SDK (endpoint determinado pela região AWS, não pela entrada do usuário), (2) leituras do Secrets Manager no cold start (nome do segredo fixo na configuração, não na entrada do usuário).                                                                                                                          |
| Consideração Fase 3  | Quando conversão de moeda for adicionada (Fase 3), o serviço buscará taxas de câmbio de um provedor de dados financeiros. Nesse ponto: a URL do provedor será uma variável de ambiente (não entrada do usuário), as requisições usarão um hostname na allowlist e as respostas serão validadas contra um schema esperado antes do uso. Esta seção deve ser atualizada antes do lançamento da Fase 3. |
| Segmentação de rede  | Funções Lambda executam na VPC gerenciada pela AWS (sem VPC do cliente para o MVP). Elas só podem alcançar serviços AWS via endpoints públicos. Sem serviços internos, bancos de dados ou endpoints de metadados acessíveis pelo Lambda nesta configuração.                                                                                                                                          |

# Requisitos de Testes

## Visão Geral da Estratégia de Testes

| Tipo de Teste                 | Obrigatório          | Meta de Cobertura                                      | Ferramentas                                 |
|-------------------------------|----------------------|--------------------------------------------------------|---------------------------------------------|
| Testes Unitários              | Sim                  | 90% de linha, 80% de branch                            | pytest + pytest-cov                         |
| Testes de Precisão Matemática | Sim                  | 100% das funções implementadas                         | pytest + hypothesis                         |
| Testes de Integração          | Sim                  | Todos os endpoints de API, interações DynamoDB         | pytest + moto (mocking AWS)                 |
| Testes de Carga               | Sim (pré-lançamento) | 1.000 requisições concorrentes, p50 < 50ms             | Locust                                      |
| Testes de Segurança           | Sim                  | Validação de entrada, prevenção de injeção             | pytest (customizado) + revisão manual OWASP |
| Testes End-to-End             | Condicional          | Jornadas críticas do usuário contra staging implantado | pytest + httpx contra API ao vivo           |

## Padrões de Testes Unitários

- **Cobertura Mínima:** 90% de cobertura de linha, 80% de cobertura de branch. Aplicado pelo `pytest-cov` com `fail_under = 90` no `pyproject.toml`.
- **Política de Mocking:** Mock de serviços AWS (DynamoDB, Secrets Manager) com `moto`. Mock de tempo com `freezegun`. Não faça mock de funções matemáticas internas. Funções matemáticas devem ser testadas com computação real.
- **Convenção de Nomenclatura:** Arquivos de teste espelham arquivos fonte. `src/calceengine/trig.py` testado em `tests/unit/test_trig.py`. Funções de teste nomeadas como `test_<função>_<cenário>` (ex.: `test_sin_zero_returns_zero`, `test_sin_negative_pi_returns_zero`).

- **Localização de Testes:** Árvore de diretório `tests/` separada. Não co-localizada com o código-fonte.

```
tests/  
  unit/  
    test_arithmetic.py  
    test_trig.py  
    test_statistics.py  
    test_expression_parser.py  
    test_error_handling.py  
  integration/  
    test_api_evaluate.py  
    test_api_trig.py  
    test_api_keys.py  
    test_rate_limiting.py  
  accuracy/  
    test_trig_accuracy.py  
    test_arithmetic_accuracy.py  
    test_statistics_accuracy.py  
  conftest.py
```

## Testes de Precisão Matemática

Esta é uma categoria de testes específica do CalcEngine que não existe na maioria dos projetos.

- **Implementação de referência:** Toda função matemática deve ser testada contra o módulo `math` do Python, a biblioteca `mpmath` (em alta precisão) ou tabelas matemáticas publicadas.
- **Testes baseados em propriedades com hypothesis:** Use hypothesis para gerar entradas válidas aleatórias e verificar que as propriedades se mantêm (ex.: `sin(x)^2 + cos(x)^2 == 1`, `log(a*b) == log(a) + log(b)`).
- **Casos extremos:** Toda função deve ter testes explícitos para: zero, zero negativo, números muito pequenos (próximos de epsilon), números muito grandes, fronteiras de domínio (ex.: `asin(1)`, `asin(1.0000001)`) e valores especiais ( $\pi$ ,  $e$ , múltiplos de  $\pi/2$  para trigonometria).
- **Tolerância:** Os resultados devem corresponder aos valores de referência dentro de 1 ULP (unit in the last place) para funções básicas. Documente quaisquer funções onde tolerância mais ampla é aceita, com justificativa.

**Exemplo de padrão de teste de precisão:**

```

import math
import pytest
from hypothesis import given, strategies as st
from calcengine.trig import sin, cos

class TestSinAccuracy:
    """Validate sin() accuracy against math.sin and known exact values."""

    @pytest.mark.accuracy
    @pytest.mark.parametrize("input_val, expected", [
        (0, 0.0),
        (math.pi / 6, 0.5),
        (math.pi / 4, math.sqrt(2) / 2),
        (math.pi / 2, 1.0),
        (math.pi, 0.0),
        (3 * math.pi / 2, -1.0),
        (2 * math.pi, 0.0),
        (-math.pi / 2, -1.0),
    ])
    def test_sin_known_values(self, input_val: float, expected: float) -> None:
        result = sin(input_val)
        assert result == pytest.approx(expected, abs=1e-15)

    @pytest.mark.accuracy
    @given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False, allow_infinity=False))
    def test_sin_matches_stdlib(self, x: float) -> None:
        assert sin(x) == pytest.approx(math.sin(x), rel=1e-15)

    @pytest.mark.accuracy
    @given(st.floats(min_value=-1e6, max_value=1e6, allow_nan=False, allow_infinity=False))
    def test_pythagorean_identity(self, x: float) -> None:
        assert sin(x) ** 2 + cos(x) ** 2 == pytest.approx(1.0, abs=1e-14)

```

## Padrões de Testes de Integração

- **Escopo:** Testar ciclo completo de requisição/resposta de API através do cliente de teste FastAPI. Testar interações DynamoDB com moto.
- **Ambiente:** Local. Sem serviços implantados necessários. `moto` mocka todos os serviços AWS.
- **Gerenciamento de Dados:** Cada teste cria sua própria tabela DynamoDB via fixture moto e desmonta após. Sem estado de teste compartilhado.



## Gates de Teste CI/CD

| Estágio do Pipeline   | Testes Obrigatórios                                                                  | Ferramentas                     | Ação em Falha                                    |
|-----------------------|--------------------------------------------------------------------------------------|---------------------------------|--------------------------------------------------|
| Pré-commit            | ruff check, ruff format --check, mypy                                                | ruff, mypy via hooks pre-commit | Bloquear commit                                  |
| Pull Request          | Testes unitários, testes de precisão, testes de integração, verificação de cobertura | pytest, GitHub Actions          | Bloquear merge                                   |
| Pré-deploy (staging)  | Todos os testes de PR + teste de carga (100 concorrentes, 60 segundos)               | pytest + Locust, GitHub Actions | Bloquear deploy                                  |
| Pós-deploy (produção) | Testes de smoke (10 cálculos representativos contra API ao vivo)                     | pytest + httpx                  | Alertar plantão. Auto-rollback se >50% de falha. |

## Executando Testes Localmente

```
# Executar todos os testes
uv run pytest

# Executar apenas testes unitários
uv run pytest tests/unit/ -m unit

# Executar apenas testes de precisão
uv run pytest tests/accuracy/ -m accuracy

# Executar com relatório de cobertura
uv run pytest --cov --cov-report=term-missing

# Executar verificação de tipos
uv run mypy src/

# Executar linter
uv run ruff check src/ tests/

# Executar verificação de formatação (sem alterações)
uv run ruff format --check src/ tests/

# Executar formatador (aplicar alterações)
uv run ruff format src/ tests/
```

# Estrutura do Projeto

```
calcengine/
  .github/
    workflows/
      ci.yml          # GitHub Actions: lint, verificação de tipo, teste em P
R
  deploy.yml         # GitHub Actions: CDK deploy no merge para main
src/
  calcengine/
    __init__.py
    main.py          # Criação do app FastAPI, handler Mangum
    config.py        # Configurações via Pydantic BaseSettings
    api/
      __init__.py
      router.py       # Router principal da API
      endpoints/
        __init__.py
        evaluate.py   # POST /v1/evaluate (avaliação de expressão)
        arithmetic.py # POST /v1/arithmetic/{operation}
        trigonometry.py # POST /v1/trigonometry/{function}
        statistics.py # POST /v1/statistics/{function}
        constants.py  # GET /v1/constants/{name}
      middleware/
        __init__.py
        auth.py       # Middleware de validação de chave de API
        rate_limit.py # Middleware de rate limiting
        request_logging.py # Logging estruturado de requisição/resposta
      models/
        __init__.py
        requests.py   # Modelos Pydantic de requisição
        responses.py  # Modelos Pydantic de resposta
        errors.py     # Modelos de resposta de erro e códigos de erro
    engine/
      __init__.py
      expression_parser.py # Tokenizador, construtor de AST, avaliador
      arithmetic.py        # Operações matemáticas básicas
      trigonometry.py      # Funções trigonométricas com validação de domínio
      statistics.py        # Funções de estatística descritiva
      constants.py         # Constantes matemáticas
      combinatorics.py     # Fatorial, permutações, combinações
      logarithmic.py       # Funções log, ln, exp
      validation.py        # Validação de entrada, verificação de domínio
      errors.py            # Tipos de exceção do domínio matemático
    storage/
      __init__.py
      dynamodb.py         # Cliente DynamoDB, operações de tabela
      api_keys.py          # CRUD de chave de API, validação, hashing
      usage.py             # Medição de uso, contadores de rate limit
      logging.py           # Configuração do structlog
  infrastructure/
    app.py                # Ponto de entrada do app CDK
```

```

stacks/
  __init__.py
  api_stack.py           # Lambda, API Gateway, domínio customizado
  data_stack.py          # Tabelas DynamoDB
  monitoring_stack.py    # Dashboards CloudWatch, alarmes
  auth_stack.py          # Pool de usuários Cognito
  docs_stack.py          # S3 + CloudFront para site de documentação
tests/
  unit/
    test_arithmetic.py
    test_trig.py
    test_statistics.py
    test_expression_parser.py
    test_combinatorics.py
    test_logarithmic.py
    test_validation.py
    test_api_keys.py
  integration/
    test_api_evaluate.py
    test_api_arithmetic.py
    test_api_trig.py
    test_api_statistics.py
    test_api_auth.py
    test_api_rate_limiting.py
  accuracy/
    test_trig_accuracy.py
    test_arithmetic_accuracy.py
    test_statistics_accuracy.py
    test_logarithmic_accuracy.py
    test_expression_parser_accuracy.py
  conftest.py            # Fixtures compartilhadas (cliente de teste FastAPI, mo
cks moto)
  pyproject.toml
  uv.lock
  .python-version        # Contém: 3.12
  .gitignore
  .pre-commit-config.yaml
  README.md

```

## Regras de Diretório

| Diretório                             | Contém                             | Regras                                                                                                                                    |
|---------------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/calceengine/</code>         | Todo o código-fonte da aplicação   | Apenas Python. Sem arquivos de configuração, sem testes, sem documentação.                                                                |
| <code>src/calceengine/engine/</code>  | Funções matemáticas puras          | Sem imports de aws, HTTP ou qualquer serviço externo. Apenas funções puras. Sem efeitos colaterais. Deve ser testável sem nenhum mocking. |
| <code>src/calceengine/api/</code>     | Rotas FastAPI, middleware, modelos | Apenas camada HTTP. Chama funções do motor. Não contém lógica matemática.                                                                 |
| <code>src/calceengine/storage/</code> | Camada de acesso DynamoDB          | Todo acesso a dados AWS isolado aqui. Sem lógica de negócio.                                                                              |
| <code>infrastructure/</code>          | Stacks CDK                         | Apenas CDK Python. Sem código de aplicação.                                                                                               |
| <code>tests/</code>                   | Todos os testes                    | Espelha estrutura <code>src/</code> . Diretórios separados <code>unit/</code> , <code>integration/</code> , <code>accuracy/</code> .      |
| <code>examples/</code>                | Código de template para padrões    | Código funcional com testes e README. Atualizado quando os padrões mudam.                                                                 |

## Código de Exemplo e Template

### Exemplo 1: Padrão de Endpoint de API

`examples/api-endpoint/README.md`:

```
# API Endpoint Pattern

## What This Demonstrates
Standard pattern for adding a new calculation endpoint to CalcEngine.
Shows: route definition, Pydantic models, engine call, error handling, logging.

## When to Use
- Adding any new calculation endpoint
- Adding any new HTTP route to the API

## When Not to Use
- Internal engine functions (see math-function example)
- Infrastructure changes (see cdk-construct example)

## Customization Guide
| Element | Customize? | Notes |
|-----|-----|-----|
| Route path and method | Yes | Follow /v1/{category}/{function} convention |
| Request/response models | Yes | Define Pydantic models specific to the endpoint |
| Engine function call | Yes | Call the appropriate engine module function |
| Error handling structure | No | Always use CalcEngineError hierarchy and error_response()
|
| Logging calls | No | Always log with request_id, api_key_id, duration_ms |
| Response envelope | No | Always return {"result": ..., "expression": ..., "computation_time_ms": ..., "engine_version": ...} |
```

```
examples/api-endpoint/example_endpoint.py:
```

```

"""Example: Standard API endpoint pattern for CalcEngine."""

import time

import structlog
from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field

from calcengine.api.middleware.auth import get_api_key_id
from calcengine.api.models.errors import error_response
from calcengine.api.models.responses import CalculationResponse
from calcengine.engine.errors import CalcEngineError
from calcengine.engine.trigonometry import sin

logger = structlog.get_logger()

router = APIRouter()

class SinRequest(BaseModel):
    """Request model for sine calculation."""

    value: float = Field(..., description="Input angle")
    angle_mode: str = Field(
        default="radians",
        pattern="^(radians|degrees)$",
        description="Angle unit: 'radians' or 'degrees'",
    )

@router.post("/v1/trigonometry/sin", response_model=CalculationResponse)
async def calculate_sin(
    request: SinRequest,
    api_key_id: str = Depends(get_api_key_id),
) -> CalculationResponse | dict:
    """Calculate the sine of the given value."""
    start = time.perf_counter()

    try:
        result = sin(request.value, angle_mode=request.angle_mode)
        elapsed = (time.perf_counter() - start) * 1000

        logger.info(
            "calculation_completed",
            endpoint="/v1/trigonometry/sin",
            input_value=request.value,
            angle_mode=request.angle_mode,
            result=result,
            computation_time_ms=round(elapsed, 3),
            api_key_id=api_key_id,
        )

    return CalculationResponse(

```

```

        result=result,
        expression=f"sin({request.value})",
        computation_time_ms=round(elapsed, 3),
    )

except CalcEngineError as e:
    elapsed = (time.perf_counter() - start) * 1000
    logger.warning(
        "calculation_failed",
        endpoint="/v1/trigonometry/sin",
        input_value=request.value,
        error_code=e.code,
        error_detail=str(e),
        computation_time_ms=round(elapsed, 3),
        api_key_id=api_key_id,
    )
    return error_response(e)

```

examples/api-endpoint/test\_example\_endpoint.py :



```

"""Example: Standard test pattern for a CalcEngine API endpoint."""

import math

import pytest
from fastapi.testclient import TestClient

from calcengine.main import app

@pytest.fixture
def client() -> TestClient:
    """Create a test client with a mocked API key."""
    return TestClient(app)

class TestSinEndpoint:
    """Tests for POST /v1/trigonometry/sin."""

    @pytest.mark.unit
    def test_sin_zero_radians(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 0, "angle_mode": "radians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        data = response.json()
        assert data["result"] == pytest.approx(0.0)
        assert "computation_time_ms" in data

    @pytest.mark.unit
    def test_sin_pi_over_2_radians(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": math.pi / 2, "angle_mode": "radians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        assert response.json()["result"] == pytest.approx(1.0)

    @pytest.mark.unit
    def test_sin_90_degrees(self, client: TestClient) -> None:
        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 90, "angle_mode": "degrees"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 200
        assert response.json()["result"] == pytest.approx(1.0)

    @pytest.mark.unit
    def test_sin_invalid_angle_mode(self, client: TestClient) -> None:

```

```

        response = client.post(
            "/v1/trigonometry/sin",
            json={"value": 1.0, "angle_mode": "gradians"},
            headers={"Authorization": "Bearer test-api-key"},
        )
        assert response.status_code == 422 # Pydantic validation error

@pytest.mark.unit
def test_sin_missing_auth(self, client: TestClient) -> None:
    response = client.post(
        "/v1/trigonometry/sin",
        json={"value": 0},
    )
    assert response.status_code == 401

```

## Exemplo 2: Padrão de Função Matemática Pura

`examples/math-function/README.md`:

```

# Math Function Pattern

## What This Demonstrates
Standard pattern for implementing a pure math function in the engine layer.
Shows: function signature, type hints, domain validation, error raising, docstring format.

## When to Use
- Adding any new mathematical function to src/calce/engine/

## When Not to Use
- API endpoints (see api-endpoint example)
- Functions that require AWS or HTTP access (those belong in api/ or storage/)

## Key Rules
- No imports from api/, storage/, or any external service
- Pure functions only: same input always produces same output
- Raise CalcEngineError subclasses for domain errors, never return None or NaN
- Type hints on all parameters and return values

```

`examples/math-function/example_function.py`:

```

"""Example: Standard pattern for a pure math function in CalcEngine engine layer."""

import math

from calcengine.engine.errors import DomainError

def log_base(value: float, base: float = 10.0) -> float:
    """Compute the logarithm of a value with the given base.

    Args:
        value: The number to compute the logarithm of. Must be positive.
        base: The logarithm base. Must be positive and not equal to 1.
            Defaults to 10 (common logarithm).

    Returns:
        The logarithm of value in the given base.

    Raises:
        DomainError: If value <= 0, base <= 0, or base == 1.
    """
    if value <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot compute logarithm of {value}",
            detail="Logarithm is only defined for positive numbers",
            parameter="value",
        )

    if base <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot use {base} as logarithm base",
            detail="Logarithm base must be positive",
            parameter="base",
        )

    if base == 1.0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message="Cannot use 1 as logarithm base",
            detail="Logarithm base 1 is undefined (division by zero in change-of-base)",
            parameter="base",
        )

    return math.log(value) / math.log(base)

```

```
examples/math-function/test_example_function.py :
```

```

"""Example: Standard test pattern for a pure math function."""

import math

import pytest
from hypothesis import given, strategies as st

from calcengine.engine.errors import DomainError
from calcengine.engine.logarithmic import log_base

class TestLogBase:
    """Tests for log_base function."""

    # --- Known values ---

    @pytest.mark.unit
    def test_log10_of_100(self) -> None:
        assert log_base(100, 10) == pytest.approx(2.0)

    @pytest.mark.unit
    def test_log2_of_8(self) -> None:
        assert log_base(8, 2) == pytest.approx(3.0)

    @pytest.mark.unit
    def test_ln_of_e(self) -> None:
        assert log_base(math.e, math.e) == pytest.approx(1.0)

    @pytest.mark.unit
    def test_log_of_1_any_base(self) -> None:
        assert log_base(1, 10) == pytest.approx(0.0)
        assert log_base(1, 2) == pytest.approx(0.0)
        assert log_base(1, math.e) == pytest.approx(0.0)

    # --- Default base ---

    @pytest.mark.unit
    def test_default_base_is_10(self) -> None:
        assert log_base(1000) == pytest.approx(3.0)

    # --- Domain errors ---

    @pytest.mark.unit
    def test_log_of_zero_raises_domain_error(self) -> None:
        with pytest.raises(DomainError, match="Cannot compute logarithm"):
            log_base(0)

    @pytest.mark.unit
    def test_log_of_negative_raises_domain_error(self) -> None:
        with pytest.raises(DomainError, match="Cannot compute logarithm"):
            log_base(-5)

    @pytest.mark.unit

```

```

def test_log_base_zero_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use 0"):
        log_base(10, 0)

@pytest.mark.unit
def test_log_base_one_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use 1"):
        log_base(10, 1)

@pytest.mark.unit
def test_log_base_negative_raises_domain_error(self) -> None:
    with pytest.raises(DomainError, match="Cannot use -2"):
        log_base(10, -2)

# --- Property-based: accuracy against stdlib ---

@pytest.mark.accuracy
@given(
    st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False),
)
def test_log10_matches_stdlib(self, x: float) -> None:
    assert log_base(x, 10) == pytest.approx(math.log10(x), rel=1e-14)

@pytest.mark.accuracy
@given(
    st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False),
)
def test_log2_matches_stdlib(self, x: float) -> None:
    assert log_base(x, 2) == pytest.approx(math.log2(x), rel=1e-14)

# --- Property-based: mathematical identity ---

@pytest.mark.accuracy
@given(
    a=st.floats(min_value=1e-100, max_value=1e100, allow_nan=False, allow_infinity=False),
    b=st.floats(min_value=1e-100, max_value=1e100, allow_nan=False, allow_infinity=False),
)
def test_log_product_identity(self, a: float, b: float) -> None:
    """log(a * b) should equal log(a) + log(b)."""
    if a * b > 0:
        assert log_base(a * b, 10) == pytest.approx(
            log_base(a, 10) + log_base(b, 10), rel=1e-10
        )

```

## Exemplo 3: Padrão de Construct CDK

`examples/cdk-construct/README.md`:

```
# CDK Construct Pattern

## What This Demonstrates
Standard pattern for defining a CDK stack for CalcEngine infrastructure.
Shows: Lambda function, API Gateway integration, DynamoDB table, IAM permissions.

## When to Use
- Adding new infrastructure resources to the project

## Key Rules
- All infrastructure in infrastructure/stacks/ directory
- One stack per logical group (api, data, monitoring, auth, docs)
- Use environment variables from CDK context, never hardcode
- Least-privilege IAM: each Lambda gets only the permissions it needs
```

examples/cdk-construct/example\_stack.py:

```

"""Example: Standard CDK stack pattern for CalcEngine."""

from aws_cdk import Duration, Stack
from aws_cdk import aws_apigatewayv2 as apigwv2
from aws_cdk import aws_dynamodb as dynamodb
from aws_cdk import aws_lambda as lambda_
from aws_cdk import aws_logs as logs
from aws_cdk.aws_apigatewayv2_integrations import HttpLambdaIntegration
from constructs import Construct

class ExampleApiStack(Stack):
    """Example stack showing Lambda + API Gateway + DynamoDB pattern."""

    def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        # DynamoDB table - single table design
        table = dynamodb.Table(
            self,
            "ExampleTable",
            partition_key=dynamodb.Attribute(
                name="PK", type=dynamodb.AttributeType.STRING
            ),
            sort_key=dynamodb.Attribute(
                name="SK", type=dynamodb.AttributeType.STRING
            ),
            billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
            encryption=dynamodb.TableEncryption.AWS_MANAGED,
            point_in_time_recovery=True,
        )

        # Lambda function
        handler = lambda_.Function(
            self,
            "ExampleHandler",
            runtime=lambda_.Runtime.PYTHON_3_12,
            handler="calcengine.main.handler",
            code=lambda_.Code.from_asset("src/"),
            memory_size=256,
            timeout=Duration.seconds(30),
            environment={
                "TABLE_NAME": table.table_name,
                "LOG_LEVEL": "INFO",
            },
            log_retention=logs.RetentionDays.ONE_MONTH,
        )

        # Grant Lambda read/write access to DynamoDB (least privilege)
        table.grant_read_write_data(handler)

        # HTTP API Gateway with Lambda integration
        api = apigwv2.HttpApi(

```

```
        self,
        "ExampleHttpApi",
        api_name="calcengine-api",
        default_integration=HttpLambdaIntegration(
            "LambdaIntegration",
            handler,
        ),
    )
```

---



## Como Este Documento Alimenta o AI-DLC

| Seção                                    | Estágio AI-DLC                      | Como É Usado                                                                            |
|------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------|
| Resumo Técnico do Projeto                | Deteção de Workspace                | Classificação Greenfield, contexto do time                                              |
| Linguagens de Programação                | Geração de Código                   | Python 3.12 aplicado, nenhuma outra linguagem sem aprovação                             |
| Padrões uv                               | Geração de Código                   | Todas as operações de dependência usam uv, pyproject.toml é fonte única de configuração |
| Frameworks e Bibliotecas                 | Geração de Código, Design NFR       | Stack FastAPI + Pydantic + Mangum, aplicação de bibliotecas proibidas                   |
| Permitidos/Não Permitidos Serviços Cloud | Design de Infraestrutura            | Apenas Lambda + API Gateway + DynamoDB para MVP                                         |
| Padrão de Arquitetura                    | Design de Aplicação                 | Monólito modular, fronteiras de módulo em engine/ vs api/ vs storage/                   |
| Padrões de Design de API                 | Design Funcional, Geração de Código | Convenções de endpoint, códigos de erro, formato de resposta                            |
| Requisitos de Segurança                  | Requisitos NFR, Design NFR          | Regras de validação de entrada, sem eval(), padrão de autenticação por chave de API     |
| Requisitos de Testes                     | Geração de Código, Build e Teste    | pytest + hypothesis, 90% de cobertura, testes de precisão obrigatórios                  |
| Estrutura do Projeto                     | Geração de Código                   | Layout exato de diretório e regras de posicionamento de arquivos                        |

| Seção             | Estágio AI-DLC    | Como É Usado                                                   |
|-------------------|-------------------|----------------------------------------------------------------|
| Código de Exemplo | Geração de Código | Padrões canônicos para endpoints, funções do motor, stacks CDK |